

# Project Journal — Dual-Mode Vision-Guided Robotic Arm

**Purpose:** Day-by-day log of decisions made, problems encountered, progress achieved, and next actions.

**Project Goal:** 6-DOF robotic arm with autonomous YOLOv8 sorting + real-time gesture teleoperation (local MediaPipe + remote MQTT).

**Workspace:** `/home/natraj/DDS` | **Stack:** ROS2 Jazzy · Gazebo Harmonic · MoveIt2 · Python

## How to Use This Journal

- Add a new entry every time you work on the project, even if progress is small
- Use the status badges: ✔ Done · 🔄 In Progress · ⌚ Pending · ✖ Blocked · 💡 Idea/Note
- Always fill in the **Next Session** section before closing — future you will thank present you
- Link to specific files, issues, or commits where relevant

## Phase Overview

| Phase | Description                                  | Status                      |
|-------|----------------------------------------------|-----------------------------|
| 0     | Robot Model & Gazebo Simulation              | ✔ Done                      |
| 1     | MoveIt2 Setup & Motion Planning              | ✔ Done (config) / 🔄 Testing |
| 2     | Local Gesture Teleoperation (MediaPipe)      | ⌚ Pending                   |
| 3     | Autonomous Sorting (YOLOv8 + MoveIt2)        | ⌚ Pending                   |
| 4     | Remote Gesture via MQTT                      | ⌚ Pending                   |
| 5     | Physical Hardware Bridge (ESP8266 + PCA9685) | ⌚ Pending                   |

## Entry 001 — Pre-Project & Hardware Sourcing

**Date:** Before March 2026  
**Phase:** 0 (Setup)  
**Time spent:** Several weeks

### What Happened

- Ordered hardware components for the 6-DOF robotic arm
- Discovered the original URDF files had component mismatches — the motors in the files were different from the actual hardware received
- Searched online for a 3D model close to the physical hardware
- Model was found in SolidWorks format (`.sldprt` / `.sldasm`) — had no SolidWorks knowledge
- Reached out to people online for help with the model conversion; waited for a response
- Tried independently — discovered **Onshape** (browser-based CAD platform)
- Uploaded the SolidWorks file to Onshape, obtained API keys
- Used VS Code + Onshape API to export the model to `.stl` and `.step` formats
- Received the exact URDF from a community member matching the physical hardware

### Outcome

- ✔ Correct URDF with matching hardware geometry obtained
- ✔ Mesh files (`.stl`) available for all 40 links
- ✔ Package `robot_description` created with `urdf/`, `meshes/`, `launch/`, `config/` structure

### Lessons Learned

- SolidWorks → Onshape → STL/STEP is a viable conversion path without CAD software
- Onshape API can be scripted from VS Code for format conversion
- Always verify URDF joint names and motor positions against physical hardware before building further

## Entry 002 — Simulation Setup & Debugging

What Was Done

Created `sim.launch.py` from scratch to launch the complete simulation pipeline:

- `robot_state_publisher` → publishes URDF + TF tree
- `gz_sim` (Gazebo Harmonic) → physics simulation
- `ros_gz_sim create` → spawns robot into world
- `ros_gz_bridge` → bridges clock, TF, joint states to ROS2
- `rviz2` → visualization

Issues Encountered and Fixed

| # | Error                                           | Root Cause                                                                  | Fix                                                                                                   |
|---|-------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1 | Physics Error Code 19 — Invalid inertia         | 16 links had <code>ixx=iyz=izz=0</code> from URDF exporter                  | Replaced zeros with small positive values (~1e-7)                                                     |
| 2 | Robot invisible — mesh URI failed               | <code>GZ_SIM_RESOURCE_PATH</code> not set for <code>model://</code> URIs    | Added <code>SetEnvironmentVariable</code> in launch file                                              |
| 3 | <code>Gazebo/Silver</code> material not found   | Ogre material strings removed in Gazebo Harmonic                            | Replaced with RGBA <code>&lt;ambient&gt;/&lt;diffuse&gt;</code> blocks                                |
| 4 | <code>libgazebo_ros_control.so</code> not found | Gazebo Classic plugin, incompatible with Harmonic                           | Removed plugin block from <code>robot.gazebo</code>                                                   |
| 5 | Mimic constraint ignored                        | DARTsim does not support <code>&lt;mimic&gt;</code> joints                  | Removed <code>&lt;mimic&gt;</code> tags from Revolute 39/40/41/44                                     |
| 6 | <code>[Dbg]</code> mesh collision spam          | DARTsim fires debug message per mesh per step                               | Changed <code>-v4</code> → <code>-v1</code> in Gazebo args                                            |
| 7 | RViz2 "jump back in time"                       | RViz2 started before Gazebo clock was stable                                | <code>TimerAction(6.0)</code> delay on RViz2 start                                                    |
| 8 | Gripper continuously rotating                   | <code>continuous</code> joints with no damping/limits after mimic removal   | Changed to <code>revolute</code> , added <code>&lt;limit&gt;</code> and <code>&lt;dynamics&gt;</code> |
| 9 | RViz2 TF errors for all gripper links           | No <code>/joint_states</code> source — RSP had no data for non-fixed joints | Added <code>JointStatePublisher</code> plugin + bridge to <code>/joint_states</code>                  |

Files Changed

- `src/robot_description/launch/sim.launch.py` — **created**
- `src/robot_description/urdf/robot.xacro` — inertia fixes, joint type fixes, dynamics added
- `src/robot_description/urdf/robot.gazebo` — materials replaced, plugin removed, JSP plugin added

Reference Docs

- Full issue details → `docs/troubleshoots/SIMULATION_ISSUES_AND_FIXES.md`

Outcome

- ☒ Robot spawns correctly in Gazebo Harmonic
- ☒ All 40 links visible with correct STL meshes
- ☒ RViz2 TF tree complete — no transform errors
- ☒ Gripper joints stable (not spinning)
- ☒ Joint states bridged from Gazebo → ROS2

Next Actions Identified

- Install Movelt2 and run Setup Assistant on the current URDF
- Create `robot_control` and `robot_vision` ROS2 packages

**Date:** March 2, 2026  
**Phase:** 0 (Simulation — TF tree cleanup)  
**Time spent:** Full session

Goal for This Session

- ✔ Analyze the messy 40-link TF tree from CAD export
- ✔ Create new `robot_arm_description` package with merged links
- ✔ Compute cumulative transforms for correct mesh placement
- ✔ Fix Gazebo JointStatePublisher (scoped name mismatch)
- ✔ Fix empty `frame_id` on `/joint_states` (RViz2 MessageFilter)

What Was Done

1. **Full kinematic chain analysis** of `robot.xacro` (1114 lines) — mapped all 40 links, 29 fixed joints, 11 actuated joints
2. **Computed cumulative transforms** using Python/numpy — verified 6 checkpoint transforms with 0.000000 error
3. **Created `robot_arm_description` package** with clean 12-link URDF:
  - 43 STL meshes reused, each link has multiple `<visual>` blocks with computed offsets
  - Proper collision primitives (boxes/cylinders) instead of mesh collision
  - Combined inertials per merged link
4. **Fixed JointStatePublisher** — removed explicit `<joint_name>` tags (scoped name mismatch in Gazebo Harmonic)
5. **Fixed empty frame\_id** — created `joint_state_relay.py` to stamp `frame_id='base_link'` on Gazebo bridge output
6. **Switched bridge config** from CLI args + remappings → YAML config file (`config/bridge.yaml`)

Issues Encountered

| #  | Issue                                                | Root Cause                                                                                                                  | Fix                                                                                                                      |
|----|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 10 | JointStatePublisher publishes nothing                | Explicit <code>&lt;joint_name&gt;</code> tags don't match Gazebo's scoped names ( <code>model::joint</code> )               | Remove all <code>&lt;joint_name&gt;</code> tags — plugin publishes ALL joints automatically                              |
| 11 | RViz2 drops every <code>/joint_states</code> message | <code>ros_gz_bridge</code> leaves <code>header.frame_id = ''</code> ; RViz2 Jazzy MessageFilter requires non-empty frame_id | Created relay node: <code>/joint_states_gz</code> → stamp <code>frame_id='base_link'</code> → <code>/joint_states</code> |

Files Created / Changed

| File                                                     | Action                                                     |
|----------------------------------------------------------|------------------------------------------------------------|
| <code>src/robot_arm_description/</code> (entire package) | <b>Created</b> — clean 12-link description                 |
| <code>urdf/robot.urdf.xacro</code>                       | New clean URDF with merged links                           |
| <code>urdf/gazebo.xacro</code>                           | Gazebo materials + JointStatePublisher (no explicit names) |
| <code>urdf/materials.xacro</code>                        | RViz material definitions                                  |
| <code>config/bridge.yaml</code>                          | YAML-based <code>ros_gz_bridge</code> config               |
| <code>scripts/joint_state_relay.py</code>                | Python relay node for frame_id fix                         |
| <code>launch/sim.launch.py</code>                        | Full simulation launch                                     |
| <code>launch/display.launch.py</code>                    | RViz2 + joint slider GUI                                   |

Reference Docs

- Full restructuring details → `docs/troubleshoots/CLEAN_URDF_AND_JOINT_STATE_FIX.md`
- Original simulation issues → `docs/troubleshoots/SIMULATION_ISSUES_AND_FIXES.md`

Outcome

- ✔ TF tree: 40 links → 12 links, 40 joints → 11 joints
- ✔ `check_urdf` passes cleanly
- ✔ `display.launch.py` works (RViz2 + joint sliders)
- ✔ JointStatePublisher publishes all 10 actuated joints
- ✔ Bridge YAML config correctly routes `/joint_states_gz` → relay → `/joint_states`
- 🔄 Joint state relay needs end-to-end simulation verification

## Next Session Plan

- Verify full `sim.launch.py` end-to-end (Gazebo + RViz2 + relay)
- Install MoveIt2 and run Setup Assistant on the new clean URDF
- Begin Phase 1: Motion Planning

---

## Entry 004 — MoveIt2 Setup & Configuration

**Date:** March 8, 2026

**Phase:** 1 — MoveIt2 Setup

**Time spent:** Full session

### Goal for This Session

- ☒ Verify simulation fully works end-to-end (Gazebo + bridge + RViz2)
- ☒ Prepare URDF for MoveIt2 (fix # in link name, generate expanded `.urdf`)
- ☒ Run MoveIt2 Setup Assistant on `robotic_arm_description`
- ☒ Configure planning groups, end effector, collision matrix
- ☒ Generate `robotic_arm_moveit_config` package
- ☒ Fix post-generation issues and get `demo.launch.py` working

### What Was Done

#### Step 1 — Pre-MoveIt URDF Preparation

1. **Renamed `Gripper_Link_Gear#1_v1_1` → `Gripper_Link_Gear1_v1_1`** in `robotic_arm.xacro`
  - The # character breaks YAML/SRDF parsing in MoveIt2 config files
  - Updated all references: link definition, joint parent/child, visual, collision, gazebo material
2. **Generated expanded `robotic_arm.urdf`** from xacro for the Setup Assistant (it requires a static `.urdf`, not `.xacro`)
  - Command: `xacro src/robot_arm_description/urdf/robotic_arm.xacro > src/robot_arm_description/urdf/robotic_arm.urdf`
  - Verified with `check_urdf`: all 12 links, 11 joints pass

#### Step 2 — MoveIt2 Setup Assistant

Launched: `ros2 launch moveit_setup_assistant setup_assistant.launch.py`

Configuration steps performed:

1. **Start** — Loaded `robotic_arm.urdf` → model loaded with 12 links, 11 joints
2. **Self-Collisions** — Generated collision matrix with 10,000 random samples → 33 pairs disabled (Adjacent/Never/Default)
3. **Virtual Joints** — Skipped (the `world_fixed` fixed joint in the URDF already anchors `base_link` to `world`)
4. **Planning Groups** — Created two groups:
  - **arm**: Joints `link_1`, `link_2`, `link_3`, `link_4`, `link_5` (kinematic chain: `base_link` → `link_5_1`)
  - **gripper**: Joint `link_6` (single actuator joint for gripper open/close)
5. **Robot Poses** — Created `home` pose: all arm joints at 0.0
6. **End Effectors** — Defined `gripper` end effector: group=`gripper`, parent link=`link_5_1`, parent group=`arm`
7. **ros2\_control** — Auto-generated `robotic_arm.ros2_control.xacro` with `mock_components/GenericSystem` hardware plugin
8. **ROS2 Controllers** — Auto-generated `ros2_controllers.yaml` with `arm` (JointTrajectoryController) + `gripper` (JointTrajectoryController) + `joint_state_broadcaster`
9. **Author Information** — Filled in
10. **Generate** — Output to `src/robotic_arm_moveit_config/` → 100% success

#### Step 3 — Post-Generation Fixes (3 Issues Found & Fixed)

##### Issue 12 — Empty controller joint arrays

The Setup Assistant generated `ros2_controllers.yaml` with empty `joints: []` arrays for both `arm` and `gripper` controllers. This is a known MoveIt2 Setup Assistant bug.

**Fix:** Manually populated the arrays:

- `arm.joints: [link_1, link_2, link_3, link_4, link_5]`
- `gripper.joints: [link_6]`
- Added `command_interfaces: [position]`, `state_interfaces: [position, velocity]`,  
`allow_nonzero_velocity_at_trajectory_end: true`

Issue 13 — Dual <ros2\_control> block conflict

Movelt's `robotic_arm.urdf.xacro` includes the expanded `robotic_arm.urdf` (which contained the `gz_ros2_control/GazeboSimSystem` hardware block from the `.trans` file) AND overlays its own `mock_components/GenericSystem` block. Two `<ros2_control>` blocks → the first one (`GazeboSimSystem`) wins → crash because that plugin isn't loadable.  
Error: `PluginlibFactory: The plugin for class 'gz_ros2_control/GazeboSimSystem' failed to load`  
**Fix:** Emptied `robotic_arm.trans` — the `ros2_control` hardware interface is now solely provided by Movelt's `robotic_arm.ros2_control.xacro` (which correctly uses `mock_components/GenericSystem` for demo mode). Regenerated the expanded `.urdf` to confirm no `<ros2_control>` element remains.

Issue 14 — Integer type mismatch in `joint_limits.yaml`

The Setup Assistant wrote `max_velocity: 1` and `max_acceleration: 0` (integer literals). The `move_group` node's parameter system expects `double` types for these fields.  
Error: `parameter 'robot_description_planning.joint_limits.link_2.max_velocity' has invalid type: Wrong parameter type, parameter is of type {double}, setting it to {integer} is not allowed`  
**Fix:** Changed all integer values to float literals (`1` → `1.0`, `0` → `0.0`) across all 6 joints in `joint_limits.yaml`.

Generated Movelt2 Config Summary

| Config File                                 | Contents                                                                                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>robotic_arm.srdf</code>               | 2 groups ( <code>arm/gripper</code> ), 1 end effector, 1 pose ( <code>home</code> ), 33 disabled collision pairs               |
| <code>ros2_controllers.yaml</code>          | <code>arm</code> (5 joints) + <code>gripper</code> (1 joint) JointTrajectoryControllers + <code>joint_state_broadcaster</code> |
| <code>joint_limits.yaml</code>              | Velocity limits per joint (1.0–2.0 rad/s), no acceleration limits                                                              |
| <code>kinematics.yaml</code>                | KDL solver for <code>arm</code> group, timeout=5s, attempts=3                                                                  |
| <code>robotic_arm.ros2_control.xacro</code> | <code>mock_components/GenericSystem</code> with position command + state interfaces for 6 joints                               |
| <code>robotic_arm.urdf.xacro</code>         | Includes expanded URDF + <code>ros2_control</code> xacro overlay                                                               |
| <code>initial_positions.yaml</code>         | All joints at 0.0                                                                                                              |

Files Created / Changed

| File                                                                    | Action                                                                                            |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>src/robotic_arm_moveit_config/</code> (entire package)            | <b>Created</b> by Setup Assistant                                                                 |
| <code>src/robotic_arm_moveit_config/config/ros2_controllers.yaml</code> | <b>Fixed</b> — populated empty joint arrays                                                       |
| <code>src/robotic_arm_moveit_config/config/joint_limits.yaml</code>     | <b>Fixed</b> — integer → float values                                                             |
| <code>src/robot_arm_description/urdf/robotic_arm.xacro</code>           | <b>Changed</b> — renamed <code>Gripper_Link_Gear#1</code> → <code>Gripper_Link_Gear1</code>       |
| <code>src/robot_arm_description/urdf/robotic_arm.trans</code>           | <b>Changed</b> — removed <code>gz_ros2_control</code> <code>ros2_control</code> block (now empty) |
| <code>src/robot_arm_description/urdf/robotic_arm.urdf</code>            | <b>Regenerated</b> — expanded URDF without <code>ros2_control</code> block                        |

Issues Encountered

| #  | Issue                                                                                   | Root Cause                                                                                                     | Fix                                                                                                                                                    |
|----|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12 | Empty <code>joints: []</code> in generated <code>ros2_controllers.yaml</code>           | Movelt2 Setup Assistant bug — doesn't populate joint lists                                                     | Manually added joint names + interfaces                                                                                                                |
| 13 | <code>gz_ros2_control/GazeboSimSystem</code> plugin load failure in demo                | Expanded URDF still had the old <code>ros2_control</code> block from <code>.trans</code> ; two blocks conflict | Emptied <code>.trans</code> file; regenerated <code>.urdf</code> ; Movelt's <code>mock_components/GenericSystem</code> is now the only hardware plugin |
| 14 | <code>Wrong parameter type {double} → {integer}</code> crash in <code>move_group</code> | Setup Assistant writes <code>max_velocity: 1</code> (int) instead of <code>1.0</code> (double)                 | Changed all values to explicit floats in <code>joint_limits.yaml</code>                                                                                |

Outcome

- ✔ `colcon` build passes for both `robotic_arm_description` and `robotic_arm_moveit_config`

- ☒ `demo.launch.py` launches successfully — all 5 nodes start:
  - `robot_state_publisher` → URDF loaded
  - `move_group` → OMPL + Pilz planners loaded, KDL kinematics initialized
  - `ros2_control_node` → `mock_components/GenericSystem` (FakeSystem) activated
  - `spawner` → `arm` + `gripper` + `joint_state_broadcaster` controllers active
  - `rviz2` → opens with Motion Planning plugin
- ☒ No errors in launch output (only expected warnings: FIFO scheduling not permitted, no octomap plugin)
- ☒ Planning groups `arm` (5 joints) and `gripper` (1 joint) correctly configured

Architecture Note — Simulation vs. Demo

There are now **two separate launch modes**:

| Mode              | Launch File                                           | Hardware Plugin                            | Joint States Source                                                                                               |
|-------------------|-------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Gazebo Simulation | <code>robotic_arm_description/sim.launch.py</code>    | None (bridge-based)                        | Gazebo<br><code>JointStatePublisher</code> →<br><code>ros_gz_bridge</code> →<br><code>/joint_states</code>        |
| MovelIt2 Demo     | <code>robotic_arm_moveit_config/demo.launch.py</code> | <code>mock_components/GenericSystem</code> | <code>ros2_control</code> FakeSystem<br>→<br><code>joint_state_broadcaster</code><br>→ <code>/joint_states</code> |

The demo mode uses fake hardware (no physics) — ideal for testing motion planning, IK, and trajectory visualization. A future "real simulation" mode will combine Gazebo + MovelIt2 (requires fixing the `gz_ros2_control` ABI issue or building from source).

Next Session Plan

- Test motion planning in RViz2: drag interactive marker → Plan → Execute
- Verify inverse kinematics with multiple goal poses
- Begin Phase 2: Create `robot_vision` and `robot_control` packages
- Write gesture teleoperation nodes (`MediaPipe` → joint angles)

Entry 005 — MovelIt2 Planning Failure Diagnosis & Gripper Fix

**Date:** March 11, 2026  
**Phase:** 1 — MovelIt2 / Motion Planning  
**Time spent:** Full session

Goal for This Session

- ☒ Diagnose why motion planning always fails ("FAILED" every attempt in RViz2)
- ☒ Fix full gripper group — only `link_6` was configured, fingers missing
- ☒ Fix critical kinematics solver misconfiguration
- ☒ Fix SRDF arm group definition for correct KDL chain resolution

Issues Found & Fixed

| #  | Issue                             | Root Cause                                                                                                      | Fix                                                                                                                                                                           |
|----|-----------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15 | Planning always returns FAILED    | <code>kinematics_solver_timeout: 0.0005</code> (5ms!) — KDL never has time to converge                          | Changed to <code>5.0</code> seconds + <code>kinematics_solver_attempts: 10</code>                                                                                             |
| 16 | KDL doesn't know end-effector tip | <code>arm</code> group defined as individual joints — KDL can't determine tip link                              | Changed to <code>&lt;chain base_link="base_link" tip_link="link_5_1"/&gt;</code> in SRDF                                                                                      |
| 17 | Gripper moves only one part       | Gripper group only had <code>link_6</code> (gear driver) — <code>link_10/link_11/link_12/link_13</code> missing | Added all 5 joints to SRDF group, <code>ros2_controllers.yaml</code> , <code>ros2_control.xacro</code> , <code>initial_positions.yaml</code> , <code>joint_limits.yaml</code> |

Files Changed

| File                                  | Change                                                                     |
|---------------------------------------|----------------------------------------------------------------------------|
| config/kinematics.yaml                | timeout: 0.005 → 5.0, added attempts: 10                                   |
| config/robotic_arm.srdf               | arm group: joint list → <chain base_link="base_link" tip_link="link_5_1"/> |
| config/robotic_arm.srdf               | Added gripper_open and gripper_closed named poses                          |
| config/robotic_arm.srdf               | Gripper group: added link_10, link_11, link_12, link_13                    |
| config/ros2_controllers.yaml          | Gripper controller: added link_10–link_13 to joints list                   |
| config/robotic_arm.ros2_control.xacro | Added hardware interfaces for link_10–link_13                              |
| config/initial_positions.yaml         | Added initial positions for link_10–link_13                                |
| config/joint_limits.yaml              | Added velocity limits for link_10–link_13                                  |

Key Concepts Clarified

- **IK/FK:** MoveIt2's KDL solver reads URDF joint origins and automatically computes forward and inverse kinematics. No manual DH table needed.
- **DH parameters:** The <origin xyz rpy> on each URDF joint is the DH transform in xyz+rpy notation. They are equivalent.
- **Planning pipeline:** OMPL samples configs → KDL checks IK → collision checker filters → valid trajectory sent to JointTrajectoryController.

Outcome

- ☒ colcon build passes for robotic\_arm\_moveit\_config
- ☒ Gripper now has all 5 joints under MoveIt2 control
- ☐ Planning fix (kinematics timeout) pending end-to-end test with demo.launch.py

Next Session Plan

- Launch demo.launch.py and verify planning succeeds (drag marker → Plan → Execute)
- Test gripper\_open / gripper\_closed named poses
- Begin Phase 2: create robot\_vision and robot\_control packages

Upcoming Tasks Backlog

Move items from here into journal entries as you work on them.

Phase 1 — MoveIt2

- ☒ Install MoveIt2 Jazzy (ros-jazzy-moveit)
- ☒ Run MoveIt Setup Assistant
- ☒ Define arm planning group (5 joints)
- ☒ Define gripper end effector
- ☒ Auto-generate self-collision matrix (33 disabled pairs)
- ☒ Generate moveit config package
- ☒ Fix post-generation bugs (empty arrays, dual ros2\_control, int → float)
- ☒ demo.launch.py launches successfully
- ☐ Test plan + execute in RViz2 (interactive marker)
- ☐ Test inverse kinematics with random goal poses

Phase 2 — Local Gesture Teleoperation

- ☐ Create ROS2 package: robot\_vision
- ☐ Create ROS2 package: robot\_control
- ☐ Install MediaPipe: pip install mediapipe
- ☐ Write camera\_node.py — reads webcam, publishes /camera/image\_raw
- ☐ Write gesture\_node.py — MediaPipe hand tracking → 21 landmarks → 6 joint angles
- ☐ Write mode\_manager\_node.py — detects hand presence → switches AUTONOMOUS ↔ LOCAL\_GESTURE
- ☐ Write joint\_commander\_node.py — receives joint targets → sends to MoveIt2 Servo
- ☐ Test: raise hand → arm follows in simulation
- ☐ Tune landmark-to-joint-angle mapping

Phase 3 — Autonomous Sorting

- ☐ Set up conveyor belt in Gazebo world (custom SDF)
- ☐ Write `object_detection_node.py` — YOLOv8 inference on camera feed
- ☐ Train or use pre-trained YOLOv8 model for target objects
- ☐ Write `depth_estimator_node.py` — 2D detection → 3D world coordinates
- ☐ Write `task_planner_node.py` — pick + sort pose generation
- ☐ Integrate with MoveIt2 for trajectory execution
- ☐ Test full pick-and-sort cycle in simulation

Phase 4 — Remote Gesture via MQTT

- ☐ Set up HiveMQ Cloud account and credentials
- ☐ Write Flask web app for mobile browser gesture capture
- ☐ Configure ngrok tunnel
- ☐ Write `mqtt_bridge_node.py` — subscribes to MQTT → publishes to ROS2 `/joint_commands`
- ☐ Update `mode_manager_node.py` to handle REMOTE\_GESTURE mode
- ☐ Test: control arm from phone browser

Phase 5 — Hardware Integration

- ☐ Flash firmware to ESP8266 (ROS2 serial bridge)
- ☐ Wire PCA9685 I2C servo driver
- ☐ Write `ros2_control` hardware interface plugin
- ☐ Map joint angles (radians) → PWM pulse widths per servo
- ☐ Calibrate servo zero positions
- ☐ Swap Gazebo controller for hardware controller in launch file
- ☐ Test all 6 joints individually
- ☐ End-to-end test: gesture → real arm movement

Decisions Log

Record any significant technical or design decisions here for future reference.

| Date     | Decision                                                                                          | Reason                                                       | Alternatives Considered                                        |
|----------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------|
| Mar 2026 | Use Gazebo Harmonic + ROS2 Jazzy                                                                  | Latest LTS combination                                       | ROS2 Humble (older)                                            |
| Mar 2026 | DARTsim physics engine                                                                            | Default for Gazebo Harmonic                                  | Bullet (requires manual setup)                                 |
| Mar 2026 | <code>revolute</code> joints for gripper fingers with ±0.5 rad limits                             | <code>continuous</code> caused free spin after mimic removal | Keeping continuous with position controller                    |
| Mar 2026 | <code>JointStatePublisher</code> Gazebo plugin over standalone <code>joint_state_publisher</code> | Publishes real physics state, not zeros                      | <code>joint_state_publisher</code> (publishes zeros only)      |
| Mar 2026 | 6s RViz2 startup delay                                                                            | Prevents "jump back in time" from clock mismatch             | <code>use_sim_time: false</code> for RViz (incorrect approach) |

Hardware Reference

| Component       | Specification  | Notes                        |
|-----------------|----------------|------------------------------|
| Microcontroller | ESP8266        | Serial link to ROS2          |
| Servo driver    | PCA9685        | 16-channel I2C               |
| Servo motors    | 6× (model TBD) | Actual units received        |
| Camera (local)  | Webcam         | MediaPipe hand tracking      |
| Camera (robot)  | TBD            | Object detection for sorting |

Environment Reference



```
# Source workspace
cd /home/natraj/DDS && source install/setup.bash

# Build specific package
colcon build --packages-select robot_description

# Launch simulation
ros2 launch robot_description sim.launch.py

# Check active topics
ros2 topic list

# Check TF tree
ros2 run tf2_tools view_frames
```

---

## Resources & References

| Topic                   | Link / Location                                                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project overview        | <a href="/home/natraj/DDS/overview.md">/home/natraj/DDS/overview.md</a>                                                                                   |
| Simulation issues doc   | <a href="/home/natraj/DDS/docs/troubleshoots/SIMULATION_ISSUES_AND_FIXES.md">/home/natraj/DDS/docs/troubleshoots/SIMULATION_ISSUES_AND_FIXES.md</a>       |
| Movelt2 Jazzy docs      | <a href="https://moveit.picknik.ai/main/index.html">https://moveit.picknik.ai/main/index.html</a>                                                         |
| ros_gz_bridge docs      | <a href="https://github.com/gazebo-sim/ros_gz">https://github.com/gazebo-sim/ros_gz</a>                                                                   |
| MediaPipe hand tracking | <a href="https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker">https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker</a> |
| HiveMQ MQTT broker      | <a href="https://www.hivemq.com/mqtt/public-mqtt-broker">https://www.hivemq.com/mqtt/public-mqtt-broker</a>                                               |
| YOLOv8 docs             | <a href="https://docs.ultralytics.com">https://docs.ultralytics.com</a>                                                                                   |

---

*Journal started: March 2, 2026*